

CS 498: Machine Perception

HW2: Point Cloud Part Segmentation

Fall 2026

Course weight	10%
Work mode	Individual
Release	Wednesday, October 7, 2026
Due	Wednesday, October 21, 2026 at 11:59 p.m. America/Chicago

1 What you will build

A chair is one object, but it is also a collection of parts: seat, back, armrests, supports, and smaller accessories. A useful 3D perception system must recognize both levels. In this homework, you will build a semantic part segmenter that assigns a label to every point in a chair point cloud.

You will compare two ways to reason about an unordered set of 3D points. PointNet processes points independently and shares information through one global summary. A Transformer instead lets every point attend directly to every other point. Both models use the same data and metrics, so you can ask a concrete question: when does learned point-to-point communication improve over one global point-set summary?

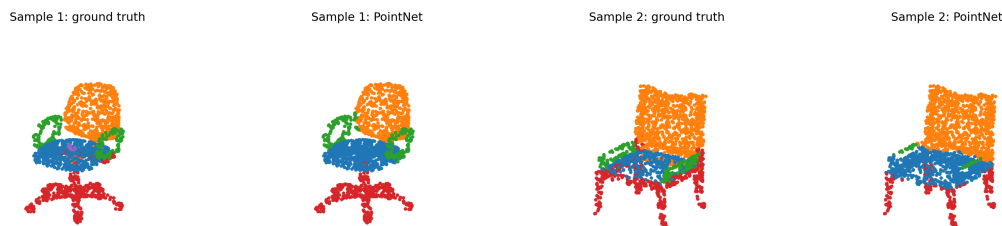


Figure 1: A reference PointNet result: ground-truth chair parts and predicted chair parts.

The assignment is based on a compact chair subset derived from PartNeXt. The course preprocessing samples a fixed number of XYZ+RGB points per object and merges the source hierarchy into five semantic labels: seat, backrest, armrest, base/support, and other/accessory. The fixed splits contain 2,400 training, 400 validation, and 600 test chairs, with 2,048 sampled surface points per chair. They are grouped by object identity to prevent the same geometry from appearing in more than one split. See the supplied data README for archive keys and provenance.

The distributed archives are deliberately small. XYZ coordinates and normals are stored as `float16`, while RGB values and labels use compact `uint8` arrays; the loader converts model inputs to `float32`. The required models use XYZ+RGB and ignore normals. Normals are included only to support the optional Utonia bonus. Labels are included in all three split archives. Do not download the full PartNeXt mesh collection for this assignment.

Representative PartNeXt chair point clouds and coarse ground-truth parts

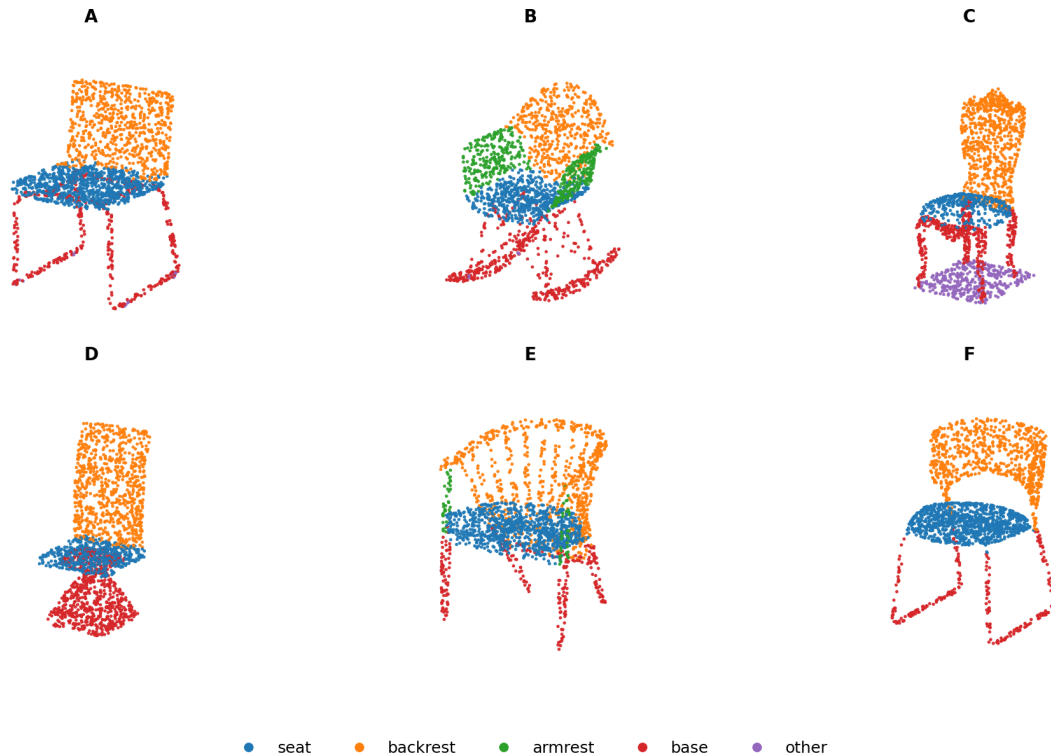


Figure 2: Representative z-up training point clouds with the five course-provided coarse ground-truth labels.

2 Run one file per task

Work in the `handout/` directory. Run each visible pipeline directly:

```
python task1_data.py
python task2_pointnet.py
python task3_transformer.py
```

Train, validation, and test discipline

Use the training split to fit model parameters. Use the validation split for every architecture, hyperparameter, epoch, and checkpoint decision; the provided pipelines save the checkpoint with the highest validation mean IoU. Only after all choices are frozen, evaluate each final required model once on the test split:

```
python task2_pointnet.py --test
python task3_transformer.py --test
```

These are the only two intended test runs: one for the final PointNet and one for the final Transformer. Test labels are distributed so these final metrics can be reproduced, but do not inspect them, choose examples from them, tune from them, or rerun training after viewing test results. All qualitative analysis and model selection must use validation data.

Edit only code between “INSERT YOUR CODE BELOW” and “DO NOT MODIFY CODE OUTSIDE THIS BLOCK”. Each graded function has its own edit block; do not modify code elsewhere. Each file’s

`main()` shows the complete pipeline and ends every subtask with a named checkpoint. Implement one block, rerun that same file, and inspect its output before continuing. Task 1 placeholders run while you work; model training waits until the Task 1 metric example passes.

File	Student work
<code>task1_data.py</code>	<code>augment_points</code> , <code>compute_confusion_matrix</code> , <code>iou_from_confusion</code>
<code>task2_pointnet.py</code>	<code>PointNetSegmenter.forward</code>
<code>task3_transformer.py</code>	<code>SelfAttention.forward</code> , <code>TransformerSegmenter.forward</code> , <code>segmentation_loss</code>

The provided `hw2_utils.py` contains low-level data loading, normalization, and plotting operations; do not edit it. NumPy, PyTorch, and Matplotlib are sufficient. Use Python 3.12 and install **requirements.txt**; the README gives environment setup and CUDA guidance. Complete Task 1 before training either model. Incomplete metric functions cause an early diagnostic instead of a long training run. Run Task 2 before Task 3; the comparison requires a saved PointNet checkpoint. Task 3 calls PyTorch’s built-in scaled dot-product attention, which selects an efficient fused CUDA kernel when the hardware and dtype support it. You do not need to install an external attention package.

3 Task 1: Data, augmentation, and evaluation [2.0 points]

1A. Inspect the point clouds

Run `task1_data.py` once before editing. The first checkpoint visualizes a chair. Each object has separate XYZ and RGB arrays with one semantic integer label per point. Checkpoint 1A writes `outputs/task1/ground_truth.png` and `outputs/task1/class_frequency.png`. Inspect both before implementing the remaining functions.

1B. Augment the point cloud

Implement `augment_points(points, rng)`. During training, apply a random rotation about the upright axis, a small isotropic scale change, and small coordinate jitter. Colors and semantic labels must remain aligned with their points. Do not rotate validation or test data. Keep the supplied function signature and return shape. Checkpoint 1B writes `outputs/task1/augmentation.png`.

1C. Confusion matrix and IoU

Implement `compute_confusion_matrix(prediction, target, num_classes)`. Use rows for ground-truth classes and columns for predicted classes. Every valid point contributes exactly once.

Then implement `iou_from_confusion(confusion)`. For class c ,

$$\text{IoU}_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FP}_c + \text{FN}_c}.$$

Return one IoU per class. If a class is absent from both prediction and target, return `NaN` for that class and exclude it from the mean. Checkpoint 1C writes `outputs/task1/confusion_matrix.txt` and `outputs/task1/metrics.json`. Confirm the supplied tiny example by hand before trusting the metric on a model.

4 Task 2: PointNet baseline [3.0 points]

PointNet treats the input as a set. The same point-wise network first maps each XYZ+RGB row to a local feature. A symmetric maximum over points creates one global feature that is unchanged by input order. Broadcast that global feature back to every point, concatenate it with the local feature, and predict one part label per point. Following PointNet [1], the segmentation path is

$$h_i = \text{MLP}([x_i, c_i]), \quad g = \max_i h_i, \quad \ell_i = \text{Classifier}([h_i, g]),$$

where x_i and c_i are the XYZ and RGB of point i , and the maximum is taken independently over every feature channel.

Required tensor convention. Let B be the batch size, $N = 2048$ the number of points, and $C = 5$ the number of part classes. The loader returns `xyz` and `rgb` as $[B, N, 3]$. The model must return raw point-aligned logits as $[B, N, C]$; point index i in the output must still describe point index i in the input.

Required architecture. Use the layers already declared in `PointNetSegmenter.__init__` exactly as shown in Figure 3. Do not add, remove, or resize layers. Every `Conv1d` has kernel size 1. Each encoder convolution is followed by batch normalization and ReLU. The first classifier convolution is followed by batch normalization, ReLU, and dropout with probability 0.2; the final convolution produces $C = 5$ raw class scores.

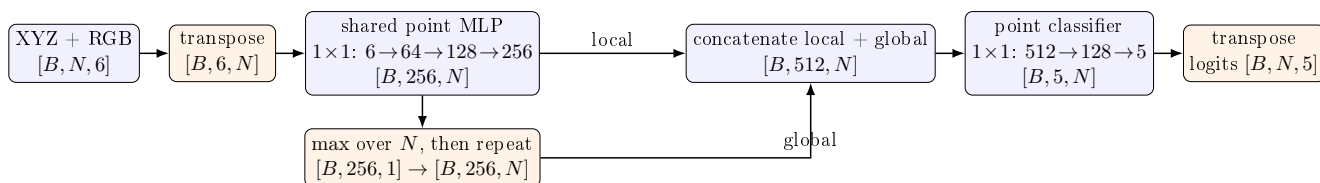


Figure 3: Required PointNet segmentation architecture. The symmetric maximum is the only operation that combines information across points. The global feature is copied to all N points before point-wise classification.

Stage	Required operation	Output shape
Input	concatenate XYZ and RGB	$[B, N, 6]$
Point encoder	transpose; 1×1 conv $6 \rightarrow 64 \rightarrow 128 \rightarrow 256$	$[B, 256, N]$
Global feature	channel-wise maximum over the N points	$[B, 256, 1]$
Local + global	repeat global feature and concatenate on channels	$[B, 512, N]$
Classifier	1×1 conv $512 \rightarrow 128 \rightarrow 5$; transpose	$[B, N, 5]$

Implementation hint. Arrange the point features as `[batch, channels, points]`. A 1D convolution with kernel size 1 then applies the same learned linear map independently to all N points. Stacking several such layers with nonlinearities implements PointNet’s shared MLP.

Implement `PointNetSegmenter.forward` in `task2_pointnet.py`. Follow the modules declared in `__init__`; do not create new trainable layers inside `forward`. The returned logits must have shape `[batch, points, classes]`. Return raw logits, not softmax probabilities, because the provided cross-entropy loss applies the normalization.

The visible `main()` trains the model, evaluates the fixed validation set, and saves these checkpoints. It runs for 100 epochs and preserves the checkpoint with the highest validation mean IoU:

- `outputs/task2/training_curves.png`;
- `outputs/task2/predictions.png`;
- `outputs/task2/metrics.json`; and
- `outputs/task2/pointnet.pt`.

The prediction figure must show at least two fixed validation samples, each with ground truth and PointNet prediction. The curve figure must show training and validation cross-entropy over epoch, plus validation mean IoU. Report the best validation epoch, validation loss at that epoch, overall point accuracy, per-class IoU, mean IoU, parameter count, and one qualitative failure. Briefly explain whether the train/validation gap suggests underfitting or overfitting. The checkpoint is for local inspection and is not submitted. After freezing this checkpoint, the final `--test` command writes `outputs/task2/test_metrics.json`; do not use that file to revise the model.

5 Task 3: Transformer [3.5 points]

PointNet’s global maximum communicates one summary of the entire shape. In this task, each point instead forms a query, key, and value and can exchange information with all other points. The input order has no meaning: the network uses XYZ features as positions and applies the same operations to every point. For one attention head, use the original Transformer definition [2]:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \quad \text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V,$$

where d is the width of one head.

Required tensor convention. Keep $B, N = 2048$, and $C = 5$ as above. The Transformer width is $W = 96$, with $H = 4$ attention heads and head width $d = W/H = 24$. All point features use shape $[B, N, W]$; attention alone temporarily uses $[B, H, N, d]$. The final output is raw logits with shape $[B, N, 5]$ in the original point order.

Required architecture. Use four identical pre-normalization Transformer blocks and dropout probability 0.1, exactly as declared in the starter. Figure 4 shows the full data flow. The learned XYZ position MLP is added to the XYZ+RGB input projection; it is not concatenated. Each block preserves $[B, N, 96]$ and contains two residual updates: multi-head self-attention, then an MLP with hidden width $4W = 384$.

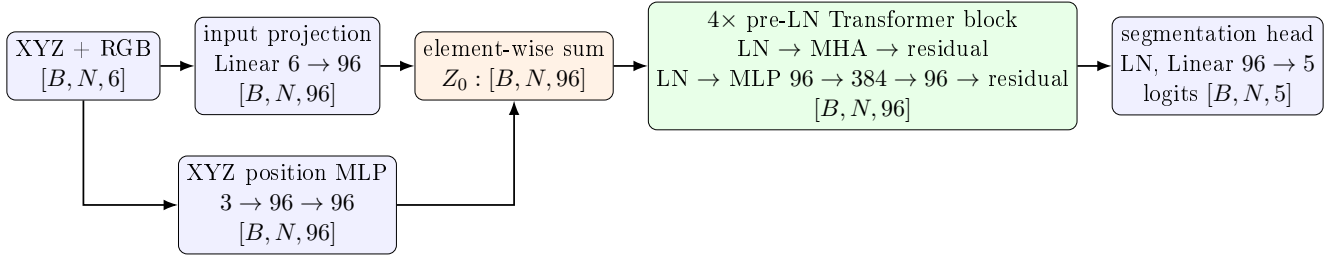


Figure 4: Required Transformer segmentation architecture. Every block preserves one feature vector per input point; only self-attention mixes information between points.

Inside each self-attention layer, the exact shape changes are:

Stage	Required operation	Output shape
Packed projection	Linear $96 \rightarrow 3 \cdot 96$	$[B, N, 288]$
Split heads	separate Q/K/V; expose 4 heads of width 24	$3 \times [B, 4, N, 24]$
Attention	<code>scaled_dot_product_attention(Q, K, V)</code>	$[B, 4, N, 24]$
Merge heads	transpose/reshape; Linear $96 \rightarrow 96$	$[B, N, 96]$
Block MLP	Linear $96 \rightarrow 384$, GELU, dropout, Linear $384 \rightarrow 96$	$[B, N, 96]$

Implementation hint. The declared `qkv` layer computes packed queries, keys, and values in one linear projection, producing shape `[batch, points, 3 * width]`. You may find `reshape` and `permute` useful for separating Q, K, V, and the attention heads.

Implement `SelfAttention.forward`. Project the input into query, key, and value tensors, split the feature channels into attention heads, and reshape each tensor to `[batch, heads, points, head_width]`. Call `torch.nn.functional.scaled_dot_product_attention`, merge the heads, and apply the declared output projection. Use the provided training-dependent dropout probability. Do not write the attention score matrix by hand.

Implement `TransformerSegmenter.forward`. Concatenate XYZ and RGB, map the six input channels with `input_projection`, add learned XYZ features from `position_mlp`, apply every declared Transformer block, and classify every point with `head`. Preserve point order and return raw logits with shape `[batch, points, classes]`.

Finally, implement `segmentation_loss(logits, labels, class_weights)` as weighted point-wise cross entropy. Flatten only the batch and point axes; do not normalize logits yourself:

$$\mathcal{L} = -\frac{\sum_i w_{y_i} \log \text{softmax}(\ell_i)_{y_i}}{\sum_i w_{y_i}}.$$

Here i ranges over all batch/point entries. Use `F.cross_entropy` with the supplied weights and `reduction="mean"`. The visible pipeline uses the same split, augmentations, optimizer family, evaluation functions, and reporting format as Task 2. It also runs for 100 epochs and preserves the checkpoint with the highest validation mean IoU. It writes:

- `outputs/task3/training_curves.png`;
- `outputs/task3/predictions.png`;
- `outputs/task3/model_comparison.png`;
- `outputs/task3/metrics.json`; and
- `outputs/task3/transformer.pt`.

Run Task 2 before Task 3. At Checkpoint 3C, the visible pipeline loads your PointNet checkpoint and evaluates both models on exactly the same fixed two validation chairs. Include the resulting two-row `model_comparison.png`: input RGB, ground truth, PointNet prediction, and Transformer prediction for each chair. Do not choose different examples for the two models. Also include the Task 3 curve with training/validation loss and validation mean IoU.

Compare PointNet and the Transformer using validation mean IoU rather than training loss alone. Discuss their curve shapes, the effect on small or thin parts, the computational tradeoff, and one visible error in the shared comparison. Full self-attention compares every pair of points, so its compute grows quadratically with point count; include this limitation in your discussion.

After freezing the Transformer checkpoint, run its final `--test` command once. It writes `outputs/task3/test_metrics.json`. Together, the Task 2 and Task 3 test files provide the two aggregate test evaluations to report; they do not produce or require test-set visualizations.

Runtime note. Use a CUDA environment for Task 3 if possible. On the staff Apple M4 MacBook Air, PointNet takes about 13 minutes for 100 epochs, but the Transformer takes about 5 minutes per epoch under MPS (roughly 8 hours for 100 epochs). The same Transformer reference run takes about 2.5 minutes on the staff CUDA workstation. Exact times depend on hardware and software versions.

6 Optional bonus: Utonia frozen linear probe [+1.0 point]

Gen-AI leverage allowed

Use an official pretrained Utonia encoder as a frozen feature extractor. Train only a point-wise linear classifier on the HW2 training split and evaluate it with the same validation metrics. The encoder must remain frozen for the entire experiment; a multilayer head or end-to-end fine-tuning does not satisfy this bonus. The goal is to measure how much a strong pretrained feature representation can improve part segmentation compared with the two models trained from scratch. The official source repository and checkpoint page are public. Its sparse-convolution dependencies may require a separate compatible environment from the Python 3.12 required tasks; cache frozen point-aligned features and stream linear-head batches if GPU memory is limited.

No bonus helper is provided. Submit `bonus_utionia.py` in addition to the four required files. Your code must identify and load the official checkpoint, reproduce its required preprocessing, extract point-aligned features, train the linear head, and save all of the following:

- `feature_pca.png`: frozen per-point features reduced to three principal components with PyTorch's PCA routine [5], rescaled to $[0, 1]$, and displayed as RGB colors for at least two fixed validation chairs. You do not need to implement PCA. This follows the DINOv2 feature-visualization convention of mapping the first three feature components to RGB [6];
- `training_curves.png`: linear-probe training/validation loss and validation mean IoU;
- `predictions.png`: qualitative predictions and ground truth for at least two fixed validation chairs; and
- `metrics.json`: best epoch, training time, trainable parameter count, validation loss, accuracy, per-class IoU, and mean IoU.

PCA component signs are arbitrary, so interpret the colors as a view of feature structure rather than semantic labels. In the report, cite the checkpoint and code source, verify that only the linear head

is trainable, and compare the probe with both required models. A runnable attempt without a valid frozen-feature evaluation receives no bonus.

7 Report and submission [1.5 points]

Use `report_template.tex` and keep the report concise (recommended 3–5 pages). Every figure must be produced by your submitted code and have a readable caption. Report the same validation definitions for both required models; do not select a different validation subset for one model.

Include both kinds of evidence: (1) a quantitative model-comparison table with validation metrics, the two final aggregate test results, parameter counts, epochs, and training time; and (2) the fixed qualitative RGB/ground-truth/PointNet/Transformer comparison on at least two validation samples, with one visible success and one visible error. Include both training-curve figures. State that the best validation mean IoU selected each checkpoint and that no model decision used the test results.

Submit exactly these required files:

1. revised `task1_data.py`;
2. revised `task2_pointnet.py`;
3. revised `task3_transformer.py`; and
4. one PDF report.

If you attempt the bonus, also submit `bonus_utonia.py`. Do not submit data, helpers, cached features, or model checkpoints. Before submission, run all three required scripts from `handout/` and confirm that your report agrees with their saved outputs.

Component	Points
Task 1: data, augmentation, and metrics	2.0
Task 2: PointNet baseline	3.0
Task 3: Transformer	3.5
Report and submission	1.5
Required total	10.0
Optional Utonia frozen linear probe	+1.0

8 Integrity and generative-AI policy

This is an individual assignment. You may discuss concepts, but the required functions, analysis, figures, and report must be your own. Generative AI tools such as GitHub Copilot, Codex, ChatGPT, Claude, and similar systems may not be used for any required task. Generative AI may be used only in the optional bonus section explicitly marked **Gen-AI leverage allowed**. Disclose any bonus use in the report, including the tool, purpose, and affected files.

Course staff may ask you to explain your implementation, use version history to understand how it developed, or make a small related modification. Report, code, saved outputs, and quantitative results must be mutually consistent. A document marker or automated detector is only a review signal and is never, by itself, evidence of misconduct.

References

[1] C. R. Qi et al., PointNet (2017). [2] A. Vaswani et al., Attention Is All You Need (2017). [3] A. Wang et al., PartNeXt (2025). [4] Pointcept, Utonia (2026). [5] PyTorch, `torch.pca_lowrank` documentation. [6] M. Oquab et al., DINOv2: Learning Robust Visual Features without Supervision (2023), Fig. 1.